

양식

2025년 한이음 드림업 개 발 보 고 서

2025. 10. 29.

프로젝트명

부족한 데이터를 보강하기 위한 제조 데이터 기반의 생성형 이미지 AI 모델 개발

 정보통신기획평가원

요 약 본

프로젝트 정보	
프로젝트명	부족한 데이터를 보강하기 위한 제조 데이터 기반의 생성형 이미지 AI 모델 개발
주제 영역	☐ 생활 ☐ 업무 ☐ 공공/교통 ☐ 금융/핀테크 ☐ 의료 ☐ 교육 ☐ 유통/쇼핑 ■ 엔터테인먼트
기술 분야	■ SW·AI ☐ 방송·콘텐츠 ☐ 블록체인·융합 ☐ 디바이스 ☐ 차세대보안 ☐ 미래통신·전파
성과 목표	■ 논문게재 및 포스터 발표 ■ 앱등록 ☐ 프로그램등록 ☐ 특허 ☐ 기술이전 ☐ 실용화 ■ 공모전(공모전명 2025 한이음 드림업 경진대회) ☐ 기타()
프로젝트 소개	본 프로젝트는 정상 이미지와 결함 마스크를 입력으로 결함 이미지를 합성하는 Stable Diffusion + ControlNet 기반 결함 합성 모델을 구축하는 것을 목표로 합니다. ControlNet과 UNet에 LoRA를 적용해 효율적인 학습을 수행하며, 제조 도메인에서 결함 데이터를 효과적으로 증강할 수 있도록 설계되었습니다.
개발 배경 및 필요성	제조 산업에서는 실제 결함 데이터를 충분히 확보하기 어렵고, 데이터 수집 비용 또한 높습니다. 이로 인해 이상 탐지 및 품질 검사 모델 학습에 필요한 결함 이미지가 부족한 문제가 발생합니다. 따라서 정상 이미지와 간단한 마스크 입력만으로도 다양한 결함 데이터를 합성할 수 있는 데이터 증강 기법이 반드시 필요합니다.
프로젝트 특징	본 프로젝트의 가장 큰 특징점은 기존 잘못된 학습 구조를 개선하여 NG latent에 노이즈를 주입하고 이를 예측하는 표준 확산 학습(ϵ/v 예측) 방식을 적용했다는 점입니다. 이를 통해 모델이 $\text{cond}(\text{mask})$ 에 기반한 결함 패턴을 보다 정확하게 학습할 수 있습니다. 또한 LoRA 기반 학습 방식을 도입하여 GPU 메모리 사용량을 최소화하면서 효율적으로 학습할 수 있으며, 학습 안정성을 위해 SNR 가중 로스, gradient clipping, 워밍업 스케줄러 등을 적용하였습니다.
주요 기능	본 프로젝트는 정상 이미지와 결함 마스크를 입력으로 받아 결함 이미지를 생성하는 기능을 중심으로 설계되었습니다. 텍스트 프롬프트 조건부 학습을 통해 클래스와 결함 유형 정보를 반영할 수 있으며, ControlNet과 Inpaint 모드를 지원하여 원본 이미지를 보존하면서 지정된 영역에만 결함을 생성할 수 있습니다.
기대효과 및 활용 분야	이 프로젝트를 통해 실제 확보하기 어려운 결함 데이터를 대량으로 합성할 수 있어 제조 산업에서의 데이터 부족 문제를 해소할 수 있습니다. 생성된 데이터를 활용하면 이상 탐지 모델의 학습 데이터가 풍부해져 모델 성능 향상에 기여할 수 있으며, 실제 스마트 제조 현장에서 발생 가능한 다양한 결함 유형을 시뮬레이션할 수 있습니다.

본 문

I. 프로젝트 개요

1. 프로젝트 소개

1) 기획 의도

본 프로젝트는 제조 산업에서 빈번히 발생하는 결함 데이터 부족 문제를 해결하기 위한 목적으로 기획되었다. 실제 산업 현장에서는 다양한 결함이 발생하지만, 그 빈도가 낮고 데이터 수집 과정에 많은 시간과 비용이 소요되기 때문에 대규모 데이터셋 확보가 쉽지 않다. 본 연구는 Stable Diffusion과 ControlNet을 활용해 이러한 결함 합성을 자동화하고, 실제 결함 이미지와 유사한 품질의 데이터를 생성할 수 있는 체계를 구축하는 것을 목표로 한다.

2) 프로젝트 내용

프로젝트는 Stable Diffusion v1.5를 기반으로 하여, ControlNet과 LoRA(Param-Efficient Fine-Tuning)를 결합한 결함 이미지 합성 파이프라인을 구축하였다. 정상 이미지와 결함 마스크(cond), 그리고 클래스·결함 유형 텍스트 프롬프트를 입력으로 받아, 결함이 반영된 NG 이미지를 합성할 수 있도록 설계하였다. 이를 통해 cond(mask)와 텍스트 프롬프트를 정확히 반영하면서 결함을 합성할 수 있으며, Inpaint 모드를 통해 원본 이미지를 보존하면서 결함 부분만을 자연스럽게 수정할 수 있다.

3) 프로젝트 정의

본 프로젝트는 정상 이미지와 결함 마스크, 그리고 텍스트 조건을 입력으로 받아 실제와 유사한 결함 이미지를 합성하는 데이터 증강용 생성 모델 개발을 의미한다. ControlNet을 통해 결함의 위치·형상을 반영하고, UNet LoRA를 통해 결함 질감을 학습하여 원본 보존 + 결함 반영이라는 두 가지 목표를 동시에 달성하는 것이 본 프로젝트의 핵심이다.

2. 개발 배경 및 필요성

1) 프로젝트 제작 동기

제조 산업에서 품질 검사를 위한 인공지능 모델을 학습하려면 대량의 결함 데이터가 필요하다. 그러나 실제 현장에서는 결함 발생 빈도가 낮고, 발생하더라도 이를 체계적으로 수집하는 데 많은 비용과 시간이 소요된다. 또한 특정 제품이나 라인에서는 일부 결함 유형만 집중적으로 발생하기 때문에, 데이터셋의 클래스 불균형 문제가 심각하다. 이로 인해 학습된 모델은 특정 결함에는 잘 동작하지만, 다른 유형의 결함에는 성능이 저하되는 한계가 존재한다.

2) 프로젝트 목적

본 프로젝트의 목적은 정상 이미지와 결함 마스크를 활용하여 실제 결함과 유사한 NG 이미지를 합성하는 데이터 증강 모델을 개발하는 것이다. 이를 통해 기존의 결함 데이터 부족 문제를 보완하고, 다양한 결함 유형을 포함하는 합성 데이터를 생성함으로써, 제조 도메인의 이상 탐지 모델과 품질 검사 자동화 시스템의 성능을 향상시키는 것을 최종 목표로 한다.

3. 프로젝트 특·장점

1) 주요 기능 관점에서의 차별성

본 프로젝트는 ControlNet과 Inpaint 모드를 동시에 지원하여 원본 이미지를 보존하면서 지정된 영역에만 결함을 자연스럽게 합성할 수 있다. 단순히 이미지를 변형하거나 합성하는 기존 기법들과 달리, 본 모델은 정상 이미지의 시각적 일관성을 유지하면서도 $\text{cond}(\text{mask})$ 와 텍스트 프롬프트 조건을 반영하여 다양한 결함 유형을 정밀하게 재현할 수 있다.

2) 유사 프로젝트와의 차별성

일반적인 이미지 합성 프로젝트나 GAN 기반 결함 생성 연구들은 결함 패턴을 데이터셋 전체 통계에 의존하여 생성하기 때문에, 특정 위치나 특정 클래스의 결함을 정밀하게 제어하기 어렵다. 반면 본 프로젝트는 $\text{cond}(\text{mask})$ 와 텍스트 프롬프트를 동시 조건으로 사용하여 결함의 위치·형상·유형을 세밀하게 통제할 수 있다. 이는 실제 제조 환경에서 특정 부품이나 특정 유형의 결함만 선택적으로 합성할 수 있는 실용적 장점을 제공한다.

3) 기존 프로젝트와 비교한 업그레이드 요소

초기 버전에서는 OK latent에 노이즈를 주입하고 NG latent 자체를 회귀하는 방식으로 학습을 수행하였으나, 이는 확산 모델의 표준 학습 방식과 어긋나 결함 반영력이 떨어지는 한계가 있었다. 본 프로젝트에서는 NG latent에 노이즈를 주입하고 해당 노이즈(ϵ 또는 velocity)를 예측하는 구조로 학습 목표를 교정하였다. 이로써 모델이 cond 와 텍스트 프롬프트 조건을 바탕으로 결함 패턴을 직접 학습하게 되어 결함 반영력이 크게 개선되었다. 또한 SNR 가중 로스를 도입하여 timestep별 학습 불균형 문제를 보완하고, gradient clipping과 learning rate warm-up 기법을 통해 학습 안정성을 강화하였다.

II. 프로젝트 내용

1. 프로젝트 구성도

프로젝트 구성은 크게 사용자 입력 → 데이터 전처리 → 생성형 AI 모델 처리 → 결과 출력의 네 단계로 이루어진다.

1) 사용자 입력 단계

사용자는 웹 인터페이스를 통해 정상(OK) 이미지와 결함 마스크(Mask), 그리고 결함 유형을 나타내는 조건 텍스트(Class명 및 Defect명)를 업로드한다. 이를 통해 모델이 생성해야 할 결함 이미지의 구체적인 조건을 명확히 정의할 수 있다.

2) 데이터 전처리 단계

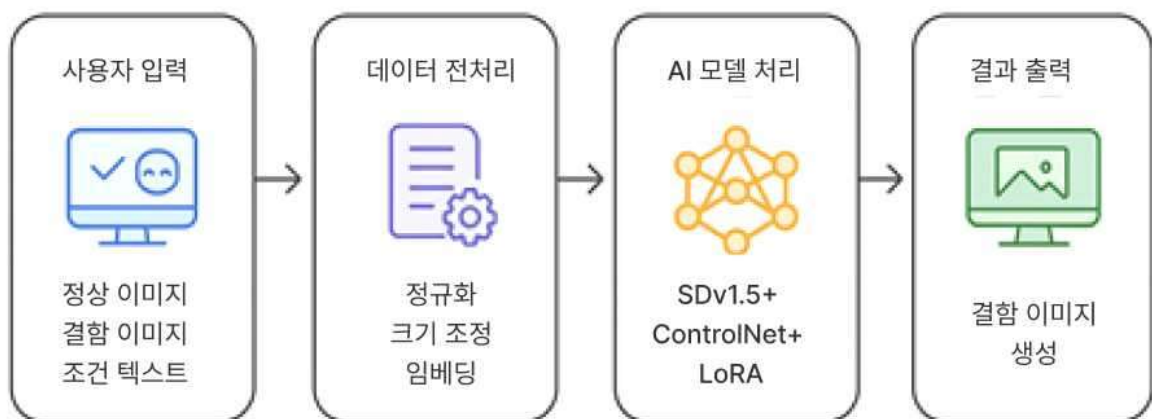
업로드된 데이터는 학습 및 추론에 적합한 형식으로 변환된다. 이미지 정규화와 크기 조정, 흑백 마스크의 3채널 변환 및 $[-1,1]$ 범위 정규화가 수행되며, 조건 텍스트는 임베딩 벡터로 변환되어 모델의 조건 입력으로 활용된다.

3) 생성형 AI 모델 처리 단계

본 프로젝트의 핵심은 Stable Diffusion v1.5를 기반으로 ControlNet과 LoRA를 결합한 생성형 AI 모델이다. ControlNet은 입력된 마스크 정보를 통해 결함의 위치와 형상을 제어하며, LoRA는 제조 도메인 결함 질감에 특화된 파인튜닝을 수행한다. 두 요소가 결합되어 정상 이미지에 특정 결함을 자연스럽게 반영한 NG 이미지를 고품질로 합성할 수 있다.

4) 결과 출력 단계

생성된 결함 이미지는 웹 인터페이스를 통해 즉시 확인할 수 있으며, 저장 및 다운로드 기능을 통해 연구용 데이터셋 구축, 제조 현장의 AI 모델 학습, 또는 결함 교육 및 시뮬레이션 기반 테스트 자료로 활용할 수 있다.



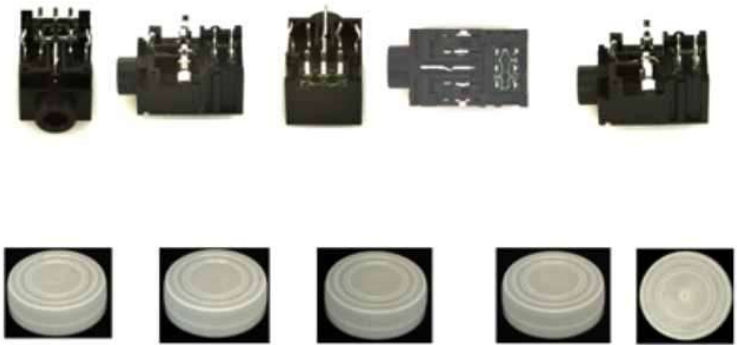
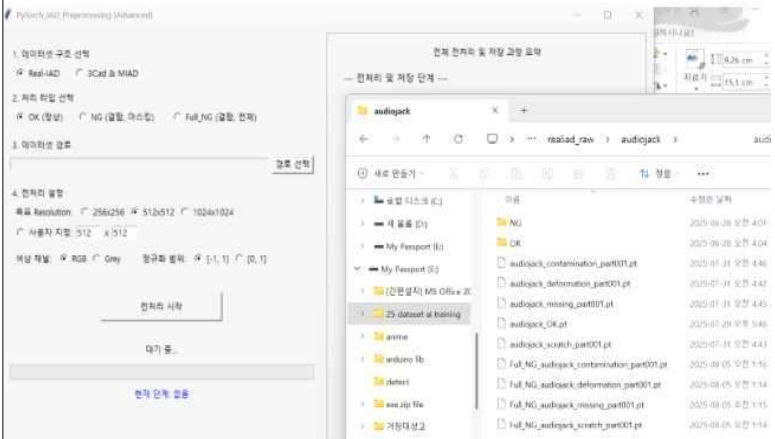
<사진 1> 서비스 흐름도

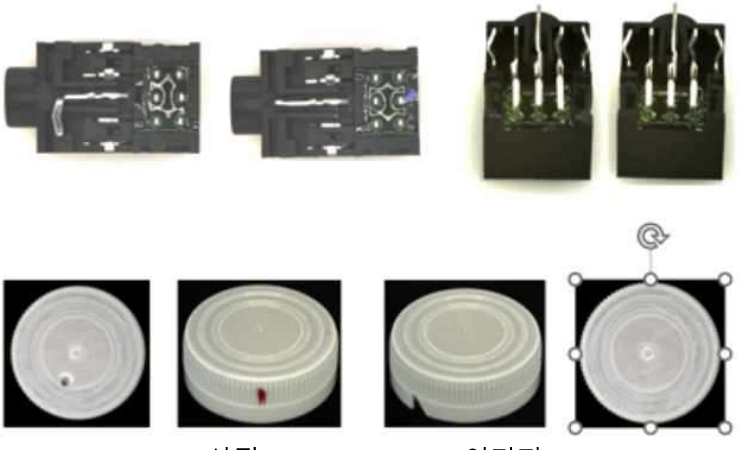
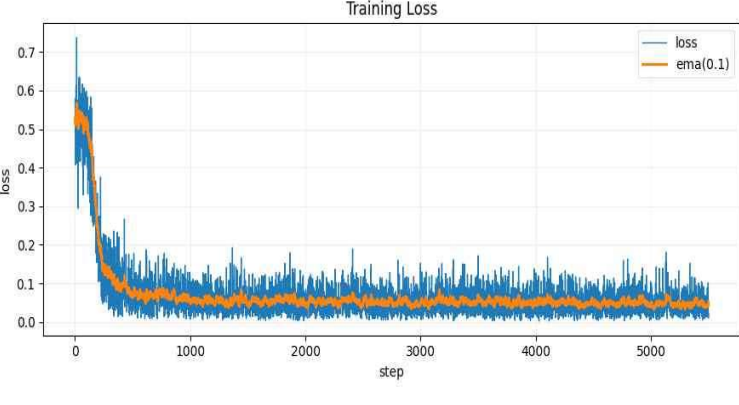
2. 프로젝트 기능

1) 전체 기능 목록

구분	기능	설명	현재진척도(%)
S/W	입력 데이터 전처리	- 정상(OK) 이미지와 결함 마스크(Mask) 업로드 - 이미지 크기 및 해상도 통일, 정규화 수행 - 이미지 텐서 변환 및 배치 구성	100%
	조건 입력 처리	- 텍스트 조건 입력 포맷 정의 (Class명, Defect명) - 결함 마스크와 텍스트 조건 매핑 로직 설계 - ControlNet 입력 조건 생성 모듈 구현 - UNet/ControlNet과의 연동 테스트 완료	100%
	결함 이미지 생성 모델 학습	- Stable Diffusion + ControlNet + LoRA 기반 모델 구조 설계 - Real-IAD 제조 데이터셋을 활용한 학습 데이터 구축 - Domain-Specific Fine-tuning 수행 (제조 결함 패턴 반영) - 학습 결과물 및 체크포인트 관리	100%
	Image-to-Image 생성 (Inference)	- OK 이미지와 결함 마스크를 입력받아 NG 이미지 합성 - 조건(Text Prompt + Mask)을 반영한 Diffusion Inference 수행 - UNet + ControlNet 모듈 최적화 및 추론 속도 개선	100%
	결과 반환 및 성능 평가	- 생성된 NG 이미지를 사용자에게 반환 (Web UI 다운로드/미리보기 제공) - FID, CLIP Score 등 자동화된 이미지 품질 지표 산출	100%
구매 S/W	클라우드 환경	- AWS Cloud: EC2 GPU 인스턴스를 활용한 모델 학습 및 추론 환경 구성 - 데이터 업로드/관리용 S3 스토리지 사용 - 확장 가능한 학습·추론 파이프라인 설계	100%
H/W	N/A		

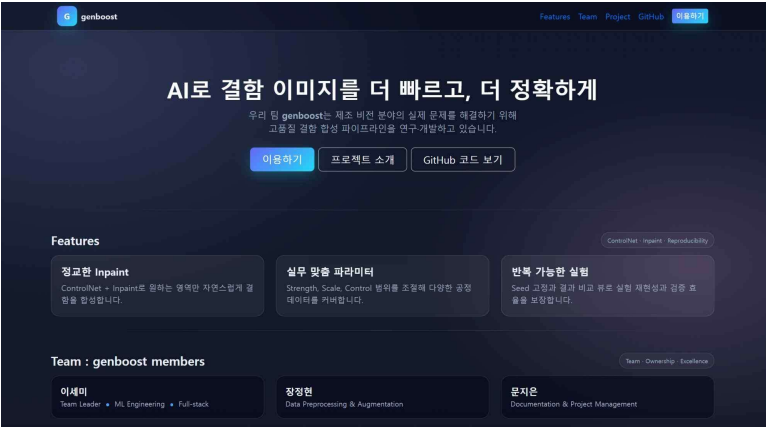
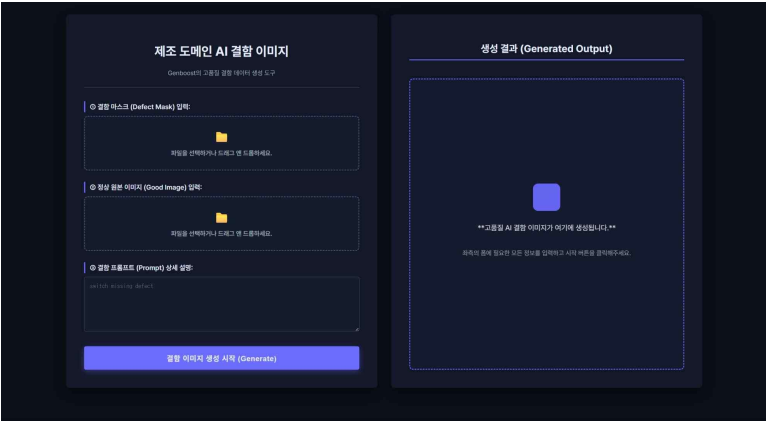
2) S/W 주요 기능

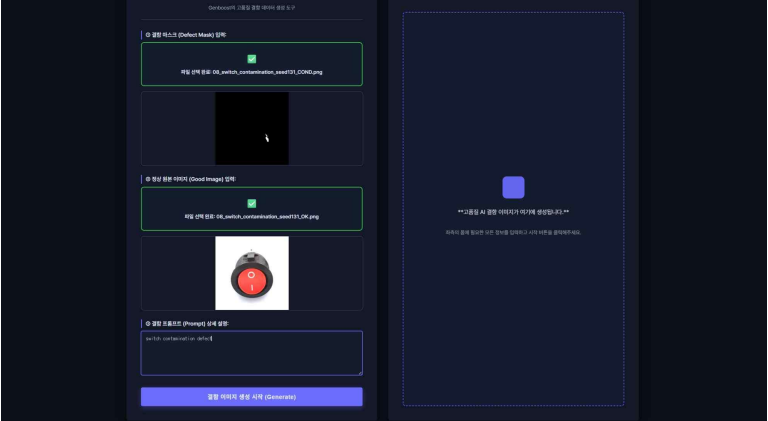
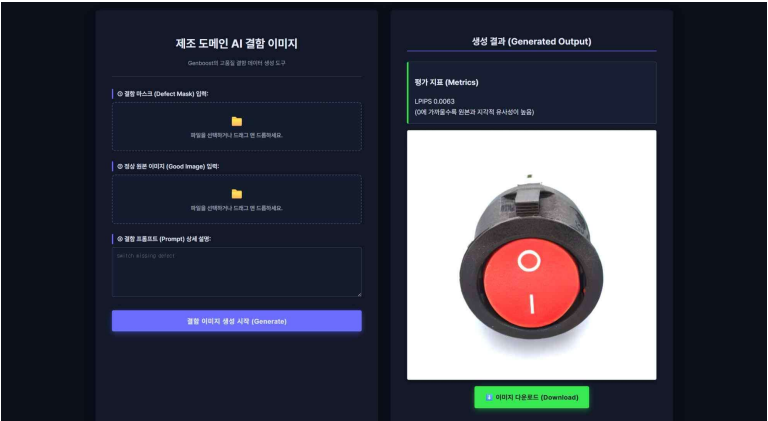
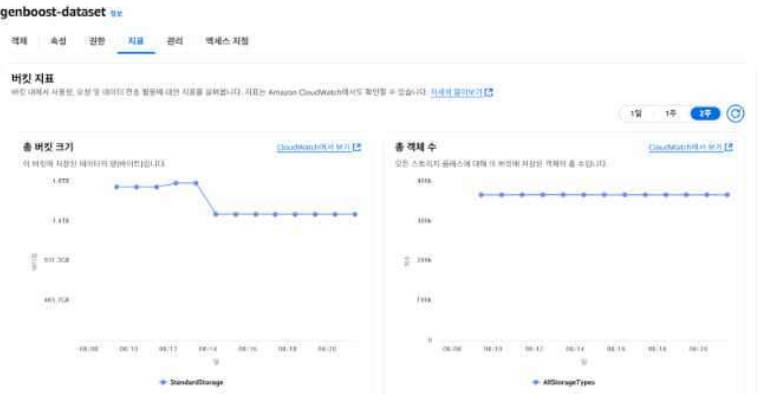
기능	설명	프로젝트실물사진
데이터 입력 및 전처리	- 정상(OK) 이미지와 결함 마스크(Mask) 업로드 - 이미지 크기 통일, 정규화, 텐서 변환 - 배치 구성 및 데이터 로더 관리	 <사진 2> 제조 공정에서 찍은 정상(OK) 제품 이미지  <사진 3> 전처리 예시  <사진 4> 데이터셋 폴더 구조 스크린샷

조건 입력 처리	• 조건 텍스트 입력 (Class명, Defect명) 관리 • Mask와 텍스트 조건 매핑 로직 설계 • ControlNet 조건 입력 모듈 구현 및 검증	 <사진 5> OK 이미지 + Mask 이미지  <사진 6> Target NG 이미지
모델 학습 (Training)	• Stable Diffusion + ControlNet + LoRA 기반 학습 구조 설계 • 제조 도메인 특화 파인 튜닝 (Fine-tuning) • 체크포인트 저장 및 모델 버전 관리	 <사진 7> 모델 학습 loss 그래프

		ubuntu@ip-172-31-33-53: ~/i2i KeyboardInterrupt (env) ubuntu@ip-172-31-33-53:~/i2i\$ python Train_re.py /home/ubuntu/env/lib/python3.10/site-packages/diffusers/utils/outputs.py:63: Use Warning: torch.utils.pytree.register_pytree_node is deprecated. Please use to ch.utils.pytree.register_pytree_node instead. torch.utils.pytree.register_pytree_node([INFO] 샘플 28,723개 클래스 18 컬럼 8 /home/ubuntu/env/lib/python3.10/site-packages/torch/utils/data/dataloader.py:558 : UserWarning: This DataLoader will create 8 worker processes in total. Our sugg ested max number of worker in current system is 4, which is smaller than what th is DataLoader is going to create. Please be aware that excessive worker creation might get DataLoader running slow or even freeze, lower the worker number to av oid potential slowness/freeze if necessary. warnings.warn(_create_warning_msg(trainable params: 3,072,120 all params: 364,351,240 trainable%: 0.84317539 80630235 Loading pipeline components...: 100% ██████████ 7/7 [00:00<00:00, 12.57it/s] Trainable params -> UNet(LoRA): 3,188,736 ControlNet(LoRA): 364,351,240 Scheduler prediction type: epsilon BNR weighting enabled (gamma=5.0) Epoch 1/2: 1% █ 99/14362 [09:10<23:03:59, 5.82s/it] [step 100] loss=0.0941 lr_ctrl=1.00e-05 lr_unet=1.00e-05 Epoch 1/2: 1% █ 112/14362 [10:00<7:20:43, 1.86s/it] ubuntu@ip-172-31-33-53: ~ 0[] 4.7% Tasks: 49, 78 thr: 2 running 1[] 12.3% Load average: 8.12 7.22 4.29 2[] 6.7% Uptime: 04:56:18 3[] 7.3% Mem[] 13.8G/15.4G Swp[] OK/OK <table><thead><tr><th>PID</th><th>USER</th><th>PR</th><th>NI</th><th>VIRT</th><th>RES</th><th>SHR</th><th>S</th><th>CPU%</th><th>MEM%</th><th>TIME+</th><th>Command</th></tr></thead><tbody><tr><td>2496</td><td>ubuntu</td><td>20</td><td>0</td><td>20.0G</td><td>2176M</td><td>74736</td><td>D</td><td>4.0</td><td>13.8</td><td>0:22.39</td><td>python Train_re.py</td></tr><tr><td>2508</td><td>ubuntu</td><td>20</td><td>0</td><td>20.0G</td><td>2178M</td><td>74856</td><td>D</td><td>4.0</td><td>13.8</td><td>0:22.65</td><td>python Train_re.py</td></tr><tr><td>2500</td><td>ubuntu</td><td>20</td><td>0</td><td>20.0G</td><td>2178M</td><td>75200</td><td>D</td><td>3.4</td><td>13.8</td><td>0:23.31</td><td>python Train_re.py</td></tr><tr><td>2504</td><td>ubuntu</td><td>20</td><td>0</td><td>20.0G</td><td>2180M</td><td>75344</td><td>D</td><td>3.4</td><td>13.8</td><td>0:23.52</td><td>python Train_re.py</td></tr><tr><td>2512</td><td>ubuntu</td><td>20</td><td>0</td><td>20.0G</td><td>2175M</td><td>74832</td><td>D</td><td>3.4</td><td>13.8</td><td>0:23.69</td><td>python Train_re.py</td></tr><tr><td>2516</td><td>ubuntu</td><td>20</td><td>0</td><td>20.0G</td><td>2189M</td><td>74464</td><td>R</td><td>3.4</td><td>13.9</td><td>0:23.28</td><td>python Train_re.py</td></tr><tr><td>2520</td><td>ubuntu</td><td>20</td><td>0</td><td>20.0G</td><td>2194M</td><td>74720</td><td>D</td><td>3.4</td><td>13.9</td><td>0:22.98</td><td>python Train_re.py</td></tr><tr><td>2524</td><td>ubuntu</td><td>20</td><td>0</td><td>20.0G</td><td>2175M</td><td>74968</td><td>D</td><td>2.7</td><td>13.8</td><td>0:23.03</td><td>python Train_re.py</td></tr><tr><td>1</td><td>root</td><td>20</td><td>0</td><td>162M</td><td>6716</td><td>3516</td><td>S</td><td>0.0</td><td>0.0</td><td>0:01.32</td><td>/sbin/init</td></tr><tr><td>144</td><td>root</td><td>19</td><td>-1</td><td>72852</td><td>4736</td><td>3840</td><td>S</td><td>0.0</td><td>0.0</td><td>0:00.31</td><td>/lib/systemd/sy</td></tr><tr><td>198</td><td>root</td><td>RT</td><td>0</td><td>346M</td><td>27520</td><td>8960</td><td>S</td><td>0.0</td><td>0.2</td><td>0:01.17</td><td>/sbin/multipath</td></tr><tr><td>201</td><td>root</td><td>20</td><td>0</td><td>346M</td><td>27520</td><td>8960</td><td>S</td><td>0.0</td><td>0.2</td><td>0:00.00</td><td>/sbin/multipath</td></tr><tr><td>202</td><td>root</td><td>RT</td><td>0</td><td>346M</td><td>27520</td><td>8960</td><td>S</td><td>0.0</td><td>0.2</td><td>0:00.00</td><td>/sbin/multipath</td></tr><tr><td>203</td><td>root</td><td>RT</td><td>0</td><td>346M</td><td>27520</td><td>8960</td><td>S</td><td>0.0</td><td>0.2</td><td>0:00.00</td><td>/sbin/multipath</td></tr></tbody></table> F1Help F2Setup F3Search F4Filter F5Free F6SortBy F7Nice F8Nice F9Kill F10Quit	PID	USER	PR	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command	2496	ubuntu	20	0	20.0G	2176M	74736	D	4.0	13.8	0:22.39	python Train_re.py	2508	ubuntu	20	0	20.0G	2178M	74856	D	4.0	13.8	0:22.65	python Train_re.py	2500	ubuntu	20	0	20.0G	2178M	75200	D	3.4	13.8	0:23.31	python Train_re.py	2504	ubuntu	20	0	20.0G	2180M	75344	D	3.4	13.8	0:23.52	python Train_re.py	2512	ubuntu	20	0	20.0G	2175M	74832	D	3.4	13.8	0:23.69	python Train_re.py	2516	ubuntu	20	0	20.0G	2189M	74464	R	3.4	13.9	0:23.28	python Train_re.py	2520	ubuntu	20	0	20.0G	2194M	74720	D	3.4	13.9	0:22.98	python Train_re.py	2524	ubuntu	20	0	20.0G	2175M	74968	D	2.7	13.8	0:23.03	python Train_re.py	1	root	20	0	162M	6716	3516	S	0.0	0.0	0:01.32	/sbin/init	144	root	19	-1	72852	4736	3840	S	0.0	0.0	0:00.31	/lib/systemd/sy	198	root	RT	0	346M	27520	8960	S	0.0	0.2	0:01.17	/sbin/multipath	201	root	20	0	346M	27520	8960	S	0.0	0.2	0:00.00	/sbin/multipath	202	root	RT	0	346M	27520	8960	S	0.0	0.2	0:00.00	/sbin/multipath	203	root	RT	0	346M	27520	8960	S	0.0	0.2	0:00.00	/sbin/multipath
PID	USER	PR	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command																																																																																																																																																																											
2496	ubuntu	20	0	20.0G	2176M	74736	D	4.0	13.8	0:22.39	python Train_re.py																																																																																																																																																																											
2508	ubuntu	20	0	20.0G	2178M	74856	D	4.0	13.8	0:22.65	python Train_re.py																																																																																																																																																																											
2500	ubuntu	20	0	20.0G	2178M	75200	D	3.4	13.8	0:23.31	python Train_re.py																																																																																																																																																																											
2504	ubuntu	20	0	20.0G	2180M	75344	D	3.4	13.8	0:23.52	python Train_re.py																																																																																																																																																																											
2512	ubuntu	20	0	20.0G	2175M	74832	D	3.4	13.8	0:23.69	python Train_re.py																																																																																																																																																																											
2516	ubuntu	20	0	20.0G	2189M	74464	R	3.4	13.9	0:23.28	python Train_re.py																																																																																																																																																																											
2520	ubuntu	20	0	20.0G	2194M	74720	D	3.4	13.9	0:22.98	python Train_re.py																																																																																																																																																																											
2524	ubuntu	20	0	20.0G	2175M	74968	D	2.7	13.8	0:23.03	python Train_re.py																																																																																																																																																																											
1	root	20	0	162M	6716	3516	S	0.0	0.0	0:01.32	/sbin/init																																																																																																																																																																											
144	root	19	-1	72852	4736	3840	S	0.0	0.0	0:00.31	/lib/systemd/sy																																																																																																																																																																											
198	root	RT	0	346M	27520	8960	S	0.0	0.2	0:01.17	/sbin/multipath																																																																																																																																																																											
201	root	20	0	346M	27520	8960	S	0.0	0.2	0:00.00	/sbin/multipath																																																																																																																																																																											
202	root	RT	0	346M	27520	8960	S	0.0	0.2	0:00.00	/sbin/multipath																																																																																																																																																																											
203	root	RT	0	346M	27520	8960	S	0.0	0.2	0:00.00	/sbin/multipath																																																																																																																																																																											
		<사진 8> 터미널 학습 진행 화면 캡처																																																																																																																																																																																				

이미지 생성 (Inference)	• OK 이미지 + Mask → NG 이미지 합성 • 조건 반영 추론 수 행 (Text Prompt + Mask 기반 I2I) • 추론 속도 최적화	OK Image Prompt: "switch contamination defect" Mask (COND) Generated NG OK Image Prompt: "regulator pit defect" Mask (COND) Generated NG <사진 9> 프롬프트 + 마스크 조건 + 생성 이미지 시각화
--------------------------	---	---

		switch contamination defect switch pit defect switch missing defect regulator contamination defect regulator pit defect regulator missing defect pcb contamination defect pcb pit defect pcb missing defect eraser contamination defect eraser pit defect eraser missing defect <사진 10> 텍스트 프롬프트 예시
시각화	• 이미지 품질 평가 지표 산출 • Django 웹 인터페이스에서 데이터 시각화	 <사진 11> 홈페이지 화면  <사진 12> 생성페이지 화면

		 <사진 13> 정상이미지, 결함 MASK 이미지 및 프롬프트 입력 화면  <사진 14> 생성된 결함 이미지 제공 화면
인프라/운영 지원	• AWS EC2 GPU 학습 및 추론 환경 구축 • S3 기반 데이터 저장 및 관리 • Web 기반 접근성 제공 (브라우저에서 사용 가능)	 <사진 15 > AWS EC2 인스턴스 관리 화면



3) H/W 주요 기능

기능/부품	설명	프로젝트실물사진
N/A		
N/A		

3. 주요 적용 기술

3.1 S/W 기능 구현을 위한 시나리오

본 프로젝트의 소프트웨어는 사용자 친화적인 프론트엔드, 안정적인 백엔드 서버, 제조 도메인에 최적화된 생성형 모델 아키텍처, 클라우드 인프라, 그리고 품질 평가 체계로 구성된다. 각 구성 요소별 시나리오는 다음과 같다.

- 프론트엔드 (Frontend)

웹 프론트엔드는 React.js 또는 Vue.js 기반의 UI 프레임워크로 개발된다. 사용자는 웹 페이지에서 정상(OK) 이미지와 결함 mask 이미지를 업로드하고, 원하는 결함 유형을 텍스트 조건으로 입력한다.

UI는 Before vs After 뷰어를 제공하여, 업로드된 원본 이미지와 모델이 생성한 결함 이미지를 직관적으로 비교할 수 있도록 한다. 또한 생성 강도, 반복 횟수 등 주요 파라미터를 조절할 수 있는 사용자 친화적 인터페이스를 제공하여 연구자 및 현업 담당자 모두가 쉽게 활용할 수 있도록 설계된다.

- 백엔드 (Backend)

백엔드는 Django 기반의 REST API 서버로 구현된다. 서버는 프론트엔드로부터 들어오는 요청을 처리하고, 입력 데이터의 유효성을 검사한 후 학습된 생성 모델을 호출한다. 사용자 업로드 파일과 생성 결과 이미지는 데이터베이스(PostgreSQL 또는 SQLite) 및 파일 관리 모듈과 연동되어 안전하게 저장된다.

- 모델 아키텍처 (Generative Model)

모델은 Stable Diffusion v1.5를 기반으로 하며, ControlNet을 추가하여 결함 mask나 참조 이미지 등 외부 조건을 활용한 구조적 제어가 가능하다.

또한 LoRA(저랭크 적응 기법)를 적용하여 제조 도메인 데이터(예: Real-IAD)를 활용한 경량화된 파인튜닝을 수행한다. 학습 전략은 정상(OK) 이미지 + 결함 Mask → 결함(NG) 이미지 합성 구조를 따른다. 이를 통해 실제 제조 현장에서 얻기 어려운 다양한 결함 데이터를 합성하고, 이상 탐지 및 품질 검사 모델의 성능을 향상시킨다.

- 인프라 및 환경 (Infrastructure)

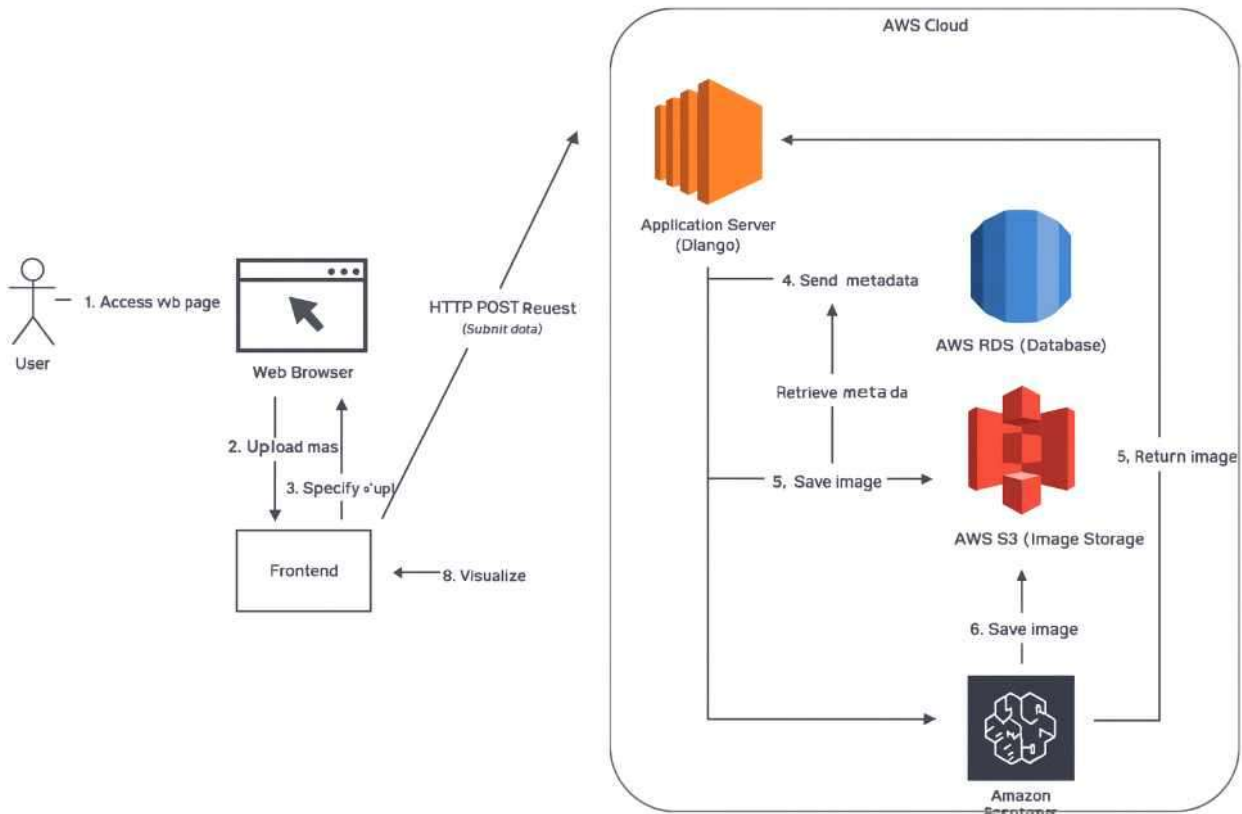
모델 학습과 추론은 AWS EC2 GPU 인스턴스를 활용하여 고성능 연산 환경에서 수행된다. 데이터는 AWS S3 스토리지에 저장되며, 학습된 모델 및 서비스는 Docker 기반 배포로 환경 일관성과 재현성을 확보한다.

또한 학습 과정 및 실험 결과는 TensorBoard, Weights & Biases(W&B) 등의 실험 관리 툴을 활용하여 시각화하고 체계적으로 관리한다.

- 품질 평가 및 관리 (Evaluation & Management)

생성된 결함 이미지의 품질은 LPIPS (Learned Perceptual Image Patch Similarity)를 통해 측정한다. 아울러, 웹 UI 상에서 사용자들이 생성된 결과 이미지를 직접 확인하고

피드백을 제공할 수 있도록 하여, 현장 적용성을 강화하고 지속적인 개선 사이클을 구축한다.



<사진 15> 시스템 아키텍처

3.2 적용 알고리즘 및 이론적 배경

본 연구에서는 Stable Diffusion v1.5를 기반으로, 결합 합성에 특화된 조건부 확산 모델(Conditional Diffusion Model) 학습 알고리즘을 적용하였다.

기존의 단순 잠재공간 회귀(Latent-to-Latent Regression) 구조는 결합 질감이나 경계 표현이 불안정하다는 한계를 보였다.

이를 해결하기 위해, 본 연구에서는 노이즈 주입 및 제거 과정을 기반으로 한 확산 확률 모델(Denoising Diffusion Probabilistic Model, DDPM) 구조를 채택하고, 결합 mask 및 OK 이미지를 조건 입력으로 통합하였다.

① Diffusion Model (Stable Diffusion v1.5 기반)

본 연구는 Stable Diffusion v1.5 구조를 기반으로 한 Denoising Diffusion Probabilistic Model (DDPM) 을 활용하였다. DDPM은 데이터에 점진적으로 가우시안 노이즈를

추가하는 전방 확산 과정(Forward Process) 과, 이를 제거하며 원본 이미지를 복원하는 역확산 과정(Reverse Process) 으로 구성된다. 즉, “이미지를 망가뜨렸다가 다시 복원하는 법”을 배우는 방식으로 작동한다.

- 전방 확산 과정(Forward Process)

정상 이미지(OK) 또는 결함 이미지(NG)의 잠재 표현(latent) x_0 에 시간 단계 t 별로 노이즈를 주입한다. 이를 식으로 표현하면 다음과 같다:

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon, \epsilon \sim N(0, I)$$

여기서 $\alpha_t = \prod_{s=1}^t (1 - \beta_s)$ 이며, β_t 는 각 단계별 노이즈 강도를 조절하는 노이즈 스케줄 값이다. 즉, 학습이 진행될수록 이미지가 점차 흐릿해지고, 결국 완전한 노이즈로 변하게 된다.

- 역확산 과정(Reverse Process)

모델의 핵심인 UNet은 위 과정에서 주입된 노이즈 ϵ (또는 속도 벡터 v)를 예측하도록 학습된다. 이 과정을 통해 모델은 “노이즈가 얼마나, 어떤 형태로 추가되었는지”를 추정하고, 그 정보를 바탕으로 결함 잠재벡터 x_t 에서 노이즈를 제거하여 원본 결함 이미지로 복원하는 방향으로 최적화된다. 결국, 모델이 노이즈 제거 능력을 익히면서 동시에 “결함이 포함된 이미지를 만드는 방법”을 학습하게 된다.

- 손실 함수

모델의 학습 목표는 예측된 노이즈 $\epsilon_\theta(x_t, t)$ 가 실제 주입된 노이즈 ϵ 와 가깝도록 하는 것이다. 손실 함수는 다음과 같이 정의된다 :

$$L = E_{t, \epsilon, x_0} [w_t \cdot ||\epsilon_\theta(x_t, t) - \epsilon||^2]$$

여기서 $w_t = \min(SNR(t), \gamma) / SNR(t)$ 은 timestep마다 다른 신호대잡음비(SNR)의 불균형을 보정하기 위한 가중치이다. 이 가중치를 적용하면, 학습 초기에 강조되는 저주파(형태 정보)와 후반부에 나타나는 고주파(세부 질감) 간의 균형 잡힌 학습이 가능해진다.

② ControlNet 기반 조건부 학습 (Conditional Diffusion)

본 연구에서는 ControlNet을 이용하여 조건부 확산 모델(Conditional Diffusion) 구조를 구현하였다. ControlNet은 기본 Stable Diffusion의 UNet 구조에 보조 네트워크를 추가하여, 입력으로 들어오는 외부 정보(예: OK 이미지, 결함 mask)를 함께 고려할 수 있도록 설계된 모델이다. 이를 통해 단순한 무조건적 생성이 아니라, 결함의 위치와 형태를 제어하며 합성할 수 있다.

- 입력 조건

- ⊙ OK 이미지 : 정상 패턴의 기본 형태를 유지하기 위한 기준 입력
- ⊙ 결합 mask / edge map : 결합의 공간적 위치 및 형태 정보를 제공하는 조건

- 작동 원리

ControlNet은 입력 조건 c (예: OK 이미지, 결합 mask 등)을 별도의 경로로 받아 이를 UNet의 각 단계(feature block)에 보조 신호(residual) 형태로 더해준다.

쉽게 말해, ControlNet은 “지도 역할을 하는 보조 네트워크”로, UNet이 이미지를 생성할 때 “어디에 결합을 넣어야 하는지”, “어떤 형태로 변형해야 하는지”를 알려주는 역할을 한다.

$$h_i' = h_i + \Delta_i(c)$$

여기서 h_i 는 UNet의 i 번째 블록의 특성 맵(feature map), $\Delta_i(c)$ 는

ControlNet으로부터 전달된 보정 신호(conditional residual)이다. 이 과정을 통해 모델은 동일한 OK 이미지로부터도 결합의 위치, 크기, 방향을 조건적으로 제어할 수 있으며, 결합이 존재하지 않는 영역에서는 원본 구조를 그대로 유지하는 안정적이고 일관된 결합 합성이 가능해진다.

③ LoRA (Low-Rank Adaptation) 기반 미세조정 (Fine-tuning)

본 연구에서는 Stable Diffusion의 UNet과 ControlNet에 LoRA(Low-Rank Adaptation) 모듈을 적용하여, 사전학습(pre-trained) 모델의 가중치를 유지하면서도 결합 특화 학습을 효율적으로 수행하였다. LoRA는 기존의 모든 가중치를 새로 학습하는 대신, 가중치 행렬 W 를 두 개의 저차원 행렬 A 와 B 로 분해하여 학습 가능한 형태로 단순화하는 방식이다.

- 원리

기존 가중치 $W \in R^{d \times k}$ 를 다음과 같이 분해한다:

$W = W + \Delta W = W + AB^T, A \in R^{d \times r}, B \in R^{k \times r}, r \ll \min(d, k)$
 학습 시에는 A, B 만 갱신하고 원본 W 는 고정(freeze)하여, 모델의 일반적 표현력을 유지하면서 도메인 특화 정보(결합 패턴)만 효율적으로 반영한다.

- 적용 위치

- ⊙ UNet : Attention 모듈(to_q, to_k, to_v, to_out.0)에 LoRA 삽입
 → 결합 질감·형태를 세밀하게 반영
- ⊙ ControlNet : 조건 인코더 경로의 Conv/Linear 계층에 LoRA 삽입
 → 결합 mask에 대한 반응 강도 조절

- 설정 및 효과

- ⊙ Rank(r) = 4, 학습 파라미터 약 1/100 수준으로 축소
- ⊙ 기존 Stable Diffusion의 사전학습 성능을 유지하면서,
- ⊙ 결합 이미지 생성에 필요한 특징만 효율적으로 학습 가능

- ⊙ GPU 메모리 사용량 약 70% 절감, 학습 수렴 속도 향상
- ⊙ UNet의 down-block은 고정(freeze), mid/up-block 중심 학습으로 결함 질감 강화

3.3 주요 적용 기술

본 개발은 기존 학습 방식의 한계(결함 미반영)를 극복하고, 모델의 결함 반영력 및 학습 효율을 극대화하기 위해 다음과 같은 핵심 기술을 적용하였다.

① 표준 확산 학습 목표로의 교정 (결함 반영 강화)

기존 Latent-to-Latent Regression 방식은 OK 이미지를 보존하려고만 하고, 결함(NG) 정보가 충분히 반영되지 않는 한계가 있었다. 이를 개선하고 결함 신호를 모델이 직접 학습하도록 교정하였다.

적용 기술 및 구현 :

- NG latent x_0 에 x_t 를 주입하고, UNet이 주입된 노이즈(ϵ) 또는 속도(v)를 예측하도록 학습 목표를 전환하였다.
- SNR(Time-dependent Signal-to-Noise Ratio) 가중치를 손실 함수에 적용하여, 학습 초기 및 후기 단계의 균형을 조정하였다.

② ControlNet 및 LoRA 기반 조건부 결함 제어

최소한의 학습 파라미터로 결함 특화 성능을 확보하고, 조건(cond)에 따른 결함 반영력을 강화하였다.

적용 기술 및 구현 :

- ControlNet LoRA (Rank 8):
 - ⊙ 결함 mask 및 edge map 정보를 인코딩하는 ControlNet 경로에 LoRA 적용
 - ⊙ 외부 조건에 대한 모델 반응 정밀도 및 강도 향상
- UNet LoRA (Rank 16, 선택적 동결):
 - ⊙ Attention 모듈(to_q, to_k, to_v, to_out.0)에 LoRA 적용 → 결함 질감 및 세부 패턴 생성

- ⊙ down_blocks LoRA는 동결, mid/up-blocks 위주 학습 → 원본 구조 보존 + 결합 디테일 강화
- 하이퍼파라미터 조정:
 - ⊙ UNet LoRA 학습률(LR) 기존 $5e-5 \rightarrow 1e-4$ 상향
 - ⊙ 결합 반영 학습 속도 증가

③ 메모리 및 속도 최적화 기술

GPU 메모리 사용량 최소화 및 연산 속도 향상을 통해 학습 효율 극대화하였다.

적용 기술 및 구현:

- 저장밀도 및 메모리 효율화:
 - ⊙ FP16: 모델 가중치 경량화 → 메모리 절반 사용, 연산 속도 향상
 - ⊙ bitsandbytes.AdamW8bit: 옵티마이저 가중치까지 8bit 처리 → 메모리 효율 극대화
- 계산 효율 최적화:
 - ⊙ xFormers attention → Attention 연산 최적화
 - ⊙ channels_last → 텐서 레이아웃 최적화
 - ⊙ ControlNet gradient checkpointing → 역전파 시 메모리 재사용
- 학습 오버헤드 최소화:
 - ⊙ 텍스트 임베딩 캐싱 → 프롬프트 인코딩 반복 제거, 학습 속도 향상

3.4 프로젝트 적용 기술 요약

항목	기술/도구	역할
모델 아키텍처	Stable Diffusion v1.5	생성 모델
학습 도구	Pytorch, Diffusers, LoRA	모델 학습 및 파인튜닝
프론트엔드	HTML, CSS, JavaScript	결과 시각화, 사용자 입력
백엔드	Django	사용자 요청 처리, 모델 호출, 데이터 관리
배포 환경	AWS EC2(g5.x2large), S3, EBS	GPU 학습, 결과 저장
실험 관리	GitHub, Jupyter Notebook	실험 코드 및 결과 관리
최적화/효율화	FP16, 8bit AdamW, xFormers attention	메모리 절감 및 연산 효율 향상
조건부 제어	ControlNet, UNet LoRA	결함 패턴 조건 반영 학습
학습 목표	DDPM ϵ/v -prediction, SNR 가중치	결함 반영력 강화, 학습 안정성 확보

4. 프로젝트 개발 환경

구분		상세내용
S/W 개발환경	OS	Ubuntu 22.04 LTS
	개발환경(IDE)	Visual Studio Code, Google Colab
	개발도구	Git, Jupyter Notebook, Conda
	개발언어	Python
	기타사항	CUDA, Hugging Face (Transformers, Diffusers), AI/ML 프레임워크 및 라이브러리(PyTorch, OpenCV, Scikit-learn 등)
프로젝트 관리환경	형상관리	GitHub
	의사소통관리	Zoom, 카카오톡, Notion
	기타사항	AWS EC2 (g5.2xlarge / A10G GPU), S3, EBS

5. 기타 사항 [본문에서 표현되지 못한 프로젝트의 가치(Value)] 및 제작 노력

1) 차별화된 성능 (Performance)

본 프로젝트는 제조 도메인에 특화된 Diffusion 기반 생성 모델을 파인튜닝하여, 기존 범용 이미지 생성 모델 대비 도메인 적합성과 조건 반영 정확도를 크게 향상시켰다. Stable Diffusion, ControlNet, LoRA와 같은 복수의 Diffusion 계열 모델을 조합·활용하여 결합 유형, 위치, 형태 등 다양한 조건을 세밀하게 반영할 수 있으며, 이를 통해 고품질 합성 결합 이미지 생성을 실현하였다.

- 이론적 근거: Diffusion 모델은 iterative denoising 과정을 통해 고해상도의 이미지를 안정적으로 생성할 수 있으며, 조건 모듈을 결합하면 특정 도메인 특화 출력이 가능하다.

2) 신뢰성 (Reliability)

학습은 AWS EC2 GPU 환경에서 수행되며, CUDA · PyTorch · Conda 기반의 환경을 고정하여 실험의 재현 가능성을 보장한다. 데이터 로딩 시 랜덤 시드를 고정하여 불필요한 변동성을 최소화하였으며, Git 기반 버전 관리 및 Weights & Biases · CSV 로그 저장을 통해 실험 과정을 체계적으로 관리하였다. 이를 통해 동일 조건에서의 재실험 및 향후 개선이 용이하다.

3) 사용성 (Usability)

사용자는 복잡한 개발 환경을 설정하지 않고도 웹 기반 인터페이스를 통해

- 이미지 업로드
- 결합 마스크 지정
- 프롬프트 입력

만으로 결과 이미지를 얻을 수 있다. 이러한 사용자 친화적 설계는 생산기술 담당자, 설계 엔지니어 등 비개발자도 직관적으로 활용할 수 있도록 하여 현업 적용성을 높였다.

4) 보안 (Security)

프로젝트 서버는 AWS EC2 환경에서 운영되며, SSH 및 Jupyter 포트 접근을 특정 IP로만 제한하였다. 또한 제조 도메인 이미지 데이터는 민감한 자산이므로 외부 접속이 차단된 폐쇄형 서버 구조로 관리하여 데이터 보안을 강화하였다.

5) 알고리즘의 창작성 (Creativity of Algorithm)

단일 알고리즘에 의존하지 않고, Mask · Sketch · Reference Image 등 복수의 조건 기반 생성 기법을 조합·실험하여 최적 구성을 탐색하였다. 이를 통해 제조 결합의 미묘한 시각적 변화를 반영할 수 있었으며, 기존 연구와 차별화되는 창의적인 알고리즘 설계를 구현하였다.

6) 데이터와 프로그램의 가치

공개 제조 데이터셋을 정제·재구성하여 조건 기반 생성에 활용 가능한 학습 포맷으로 변환하였다. 학습 로그, 평가 지표, 결과 이미지, 전처리 코드까지 통합 관리되어 있어, 향후 다른 연구자와 조직에서도 재사용 가능한 연구 자산으로서 가치를 지닌다.

7) 프로젝트를 통한 가치 창출

코드 구조를 Plug-and-Play 방식으로 설계하여, 새로운 생성 모델이나 조건 모듈을 쉽게 추가할 수 있다. 이를 통해 데이터 부족 문제 해결에 직접 기여하며, 다양한 제조 환경으로 확장 가능한 범용 기술 자산으로 발전할 수 있다.

III. 프로젝트 수행 내용

1. 프로젝트 수행일정

프로젝트 기간 (한이음 드림업 사이트 기준)		2025. 4. 1. ~ 2025. 10. 31.							
구분	추진내용	프로젝트 기간							
		3월	4월	5월	6월	7월	8월	9월	10월
계획	프로젝트 개요 및 핵심 기술 검토								
분석	프로젝트 주요 기능 적용 방식 검토								
설계	데이터셋 수집								
	학습 모델 선정								
개발	모델 학습 및 최적화								
	학습 구조 개선 및 경량화								
	시스템 구현 및 통합								
테스트	생성 이미지 품질 평가 및 성능 검증								
종료 (성과등록)	최종 보고서 작성 및 공모전 등록								

2. 프로젝트 추진 과정에서의 문제점 및 해결방안

1) 프로젝트 관리 측면

- 문제점 : 팀원들의 기술적 역량과 경험 수준이 다양하여, 프로젝트 진행 속도가 일정하지 않았고, 초기에는 서로의 의도를 충분히 이해하는 데 시간이 소요되었다. 이로 인해 의사소통 과정에서 혼선이 발생하고 업무 분담에도 어려움이 있었다.
- ▷ 해결 방안 : 정기적인 스터디 및 기술 공유 세션을 운영하여 팀원 간 지식 격차를 줄이고, 공통 이해도를 높였다. 또한 팀원들의 경험과 강점을 고려한 역할 분담을 통해 업무 효율성을 극대화하였다.

2) 프로젝트 개발 측면

- 문제점 1 : 프로젝트 초기에는 Diffusion 모델을 처음부터 직접 학습하는 방향으로 설계했으나, 고성능 GPU 및 대규모 자원이 필요하다는 현실적 제약에 직면하였다.
- ▷ 해결 방안 1 : 전략을 변경하여, 사전 학습된 Stable Diffusion 모델을 기반으로 필요한 부분만 미세 조정(Fine-tuning) 하는 방식으로 접근하였다. 이를 통해 자원 소모를 최소화하면서도 제조 도메인에 적합한 결과를 도출할 수 있었다.
- 문제점 2 : 결함 이미지를 생성하는 과정에서, 실제 제조 현장에서 발생하는 미세하고 복잡한 결함 표현이 제대로 구현되지 않았다. 단순히 결함 이미지만 학습하는 방식으로는 작은 스크래치나 불규칙적인 변형을 자연스럽게 생성하는 데 한계가 있었다.
- ▷ 해결 방안 2 : 학습 데이터셋 구성 방식을 개선하였다. 구체적으로는 정상(OK) 이미지와 결함(NG) 이미지의 쌍(pair)을 활용하여 모델이 두 클래스 간의 차이를

학습하도록 하였고, 결함 Mask 이미지를 함께 제공하여 특정 위치·형태의 결함을 조건으로 반영할 수 있도록 하였다. 이를 통해 모델은 보다 현실적이고 세밀한 결함을 생성할 수 있었으며, 실제 제조 라인의 결함 특성과 유사한 품질의 데이터를 확보할 수 있었다.

- 문제점 3 : DataLoader worker 수와 prefetch factor를 높였을 때, 시스템 RAM이 한계치에 도달하여 학습 속도가 저하되거나 OOM(Out-of-Memory) 오류가 발생하였다.
 - ▷ 해결 방안 3 : DataLoader 파라미터를 num_workers=2~4, prefetch_factor=1~2로 줄여 메모리 부담을 완화하였다. 더하여 persistent_workers=False로 설정해 epoch 종료 시 메모리를 회수하도록 개선하였고, pin_memory 옵션을 활성화하여 GPU 전송 속도를 개선하면서 메모리 효율성을 유지하였다.
- 문제점 4 : 메모리 제한으로 인해 worker 수를 줄이면 학습 속도가 느려지는 trade-off가 발생하였다.
 - ▷ 해결 방안 4 : xFormers와 SDPA(Self-Attention Optimization)를 비교 적용해 가장 빠른 백엔드를 고정하였다. GPU에서 bf16 연산이 가능할 경우 FP16 대신 bf16으로 전환하여 연산 안정성과 속도를 동시에 확보하였다.
- 문제점 5 : 본 프로젝트의 초기 학습은 OK 이미지에 노이즈를 주입한 뒤 NG 이미지 자체를 회귀하는 구조를 사용하였다. 그러나 이는 확산 모델의 표준 학습 방식(노이즈 예측: ϵ -prediction 또는 v-prediction)과 어긋나는 접근으로, 모델이 결함 패턴을 효과적으로 학습하지 못하고 원본 이미지는 보존하면서도 결함이 희미하게 반영되는 한계가 나타났다.
 - ▷ 해결방안 5 : 학습 목표를 NG 이미지 + 노이즈 주입 $\rightarrow$ 해당 노이즈(ϵ) 또는 velocity(v) 예측으로 교정하였다. 이 방식은 확산 모델의 이론적 표준에 맞추어, ControlNet과 UNet이 “주어진 마스크 조건 및 텍스트 프롬프트일 때 어떤 결함 패턴을 주입해야 하는가”를 직접적으로 학습하도록 한다. 또한 SNR 가중 손실과 LoRA 최적화 구성을 함께 적용하여 결함 반영력과 수렴 안정성을 동시에 확보하였다. 추론 단계에서는 Inpaint + ControlNet 세팅을 결합하여, 원본 이미지를 최대한 보존하면서도 지정된 결함 영역(mask)이 선명하고 뚜렷하게 표현되도록 파이프라인을 설계하였다. 이를 통해 “원본 보존 + 결함 강조”라는 프로젝트 목표를 달성할 수 있었다.

3. 프로젝트를 통해 배우거나 느낀 점

이번 프로젝트를 통해 제조 도메인 특화 생성형 AI 모델 개발의 실제적인 어려움과 가능성을 직접 체험할 수 있었다. 우선, 불량(NG) 데이터의 부족이라는 현실적인 문제를 해결하기 위해 생성형 모델을 활용하는 전략을 적용하면서, 데이터 증강의 새로운 가능성을 확인하였다. 단순히 데이터를 늘리는 것을 넘어, 결함의 위치·형태·강도를 제어할 수 있는 조건 기반 합성이 품질 검사 모델의 성능 향상에 실질적인 도움을 줄 수 있음을 배웠다.

개발 과정에서 발생한 loss NaN 문제, 메모리 한계, 결함 반영 부족 현상 등은 처음에는 큰 난관으로 느껴졌지만, 학습 구조를 표준 확산 방식에 맞게 교정하고 다양한 실험을 반복하면서 문제 해결 능력을 키울 수 있었다. 특히, 단순히 모델 아키텍처를 가져다 쓰는 것이 아니라 왜 이론적으로 맞지 않았는지, 어떤 방식으로 개선해야 하는지를 이해하는 과정이 매우 의미 있었다.

또한, 프론트엔드·백엔드·모델·인프라가 유기적으로 연결되어야 실질적인 서비스가 가능하다는 점을 깨달았다. 단순히 모델 성능만 높이는 것이 아니라, 사용자 친화적 UI, 안정적인 서버 운영, 데이터 보안까지 고려해야 프로젝트가 현장에 적용될 수 있다는 것을 배웠다.

무엇보다 이번 경험을 통해, 생성형 AI가 단순한 이미지 생성 도구가 아니라 실질적인 산업 문제 해결 수단으로 확장될 수 있다는 확신을 얻었다. 앞으로는 다양한 제조 환경과 도메인에 맞게 기술을 확장·적용하는 것이 중요한 과제임을 느꼈다.

IV. 기대효과 및 활용분야

1. 프로젝트의 기대효과

본 프로젝트를 통해 제조 공정에서의 데이터 부족 문제를 효과적으로 해결할 수 있다. 특히 실제 불량 데이터를 확보하기 어려운 환경에서도, 생성형 AI 기반 데이터 증강 기법을 적용함으로써 다양한 결함 유형을 반영한 고품질 학습 데이터셋을 구성할 수 있다. 이를 통해 다음과 같은 효과가 기대된다 :

- AI 기반 불량 탐지 정확도 향상

기존 데이터셋의 한계를 극복하여 모델 학습의 다양성과 일반화를 강화함으로써, 실제 제조 환경에서의 검출 성능을 높일 수 있다.

- 데이터 구축 비용·시간 절감

기존에 반복되던 대량 데이터 수집 및 라벨링 과정에서 발생하는 막대한 비용과 시간이 절약되며, 효율적인 데이터 자산 구축이 가능하다.

- 품질 관리 및 생산성 향상

본 프로젝트의 모델을 기반으로 한 품질 검사 시스템을 구축할 경우, 불량을 감소 및 생산 공정 효율성 향상으로 이어질 수 있다.

- 산업 현장의 디지털 전환 기여

생성형 AI 기반 결함 데이터 생성 기술은 제조업 전반의 스마트팩토리 구현과 AI 기반 품질 관리 수준 향상에 기여할 수 있다.

2. 프로젝트의 활용분야

본 프로젝트에서 개발된 결함 이미지 합성 기반 제조 AI 기술은 특정 제품군이나 산업군에 한정되지 않으며, 다양한 제조 및 비전 AI 응용 분야에서 폭넓게 활용될 수 있다. 이 기술은 데이터 확보가 어려운 제조 현장에서 결함 이미지를 인공적으로 생성하고, 불균형한 학습 데이터를 보완함으로써, AI 기반 품질 검사 시스템의 정확도와 실용성을 향상시키는 데 기여할 수 있다.

주요 활용 가능 분야는 다음과 같다:

1) 제조 산업 현장

본 기술은 제조 공정 전반의 불량 검출, 설계 검증, 품질 관리 등에 직접 적용 가능하다.

- 외관 검사 자동화 강화

실제 생산 라인에서 발생하기 어려운 다양한 결함 유형(스크래치, 오염, 균열, 이물질 등)을 합성하여 검사 알고리즘이 다양한 패턴을 학습할 수 있도록 한다. 이를 통해 AI 비전 시스템의 오탐(False Positive) 및 미탐(False Negative) 발생률을 낮출 수 있다.

- 예지보전 및 불량 예측 시스템 고도화

합성된 결함 데이터를 활용해 생산 설비 이상 및 품질 저하 패턴을 사전에 예측함으로써, 설비 유지보수 비용 절감 및 생산 안정성 향상에 기여할 수 있다.

2) 비전 AI 데이터 증강 도구

본 기술은 AI 데이터 확보가 어려운 산업군에서 데이터 확충 및 불균형 해소를 위한 데이터 증강 도구로 활용될 수 있다.

- 데이터 확보 비용 절감

실제 불량 샘플은 발생 빈도가 낮고 촬영 환경 제약이 많다. 본 기술을 활용하면 동일한 정상(OK) 이미지를 기반으로 다양한 형태의 결함 이미지를 생성하여, 데이터 수집 및 라벨링 비용을 크게 절감할 수 있다.

- 희귀 결함 패턴 생성

실제 현장에서 거의 발생하지 않지만, 품질 관리 측면에서 중요성이 높은 희귀 결함(rare defect)을 인공적으로 생성함으로써, AI 모델의 범용성 및 강건성(robustness)을 강화할 수 있다.

- 산업별 적용 확장성

자동차 부품(스크래치, 표면 오염), 전자기기(PCB 불량, 납땜 결함), 반도체(웨이퍼 결함, 포토리소그래피 결함) 등 비전 기반 품질검사가 핵심인 모든 산업 분야에 적용 가능하다.

3) 스마트팩토리 및 산업 자동화 시스템

스마트팩토리 구축 시, AI 기반 품질 검출 모듈은 전체 자동화 효율의 핵심 요소이다. 본 기술은 이러한 산업 자동화 인공지능의 신뢰성 및 안전성 향상에 직접 기여할 수 있다.

- 생산 라인 설계 및 시뮬레이션 단계 활용

실제 데이터를 수집하기 전, 다양한 가상 결함 데이터를 생성해 AI 품질 검사 모듈을 사전 학습 및 검증할 수 있다. 이를 통해 자동화 설계 초기 단계부터 리스크 없는 알고리즘 튜닝이 가능하다.

- AI 검사 시스템의 성능 유지보수

주기적으로 새롭게 생성된 결함 데이터를 활용하여 AI 시스템을 재학습시킴으로써, 생산 환경 변화(조명, 시점, 노이즈 등)에 따른 성능 저하를 최소화할 수 있다.

4) 연구 및 교육 분야

본 기술은 산업 AI 연구뿐 아니라 AI 교육, 데이터셋 구축, 모델 검증 플랫폼 등 학술적 영역에서도 높은 활용 가치를 가진다.

- 제조 AI 연구용 오픈 데이터셋 구축

실제 산업 데이터를 기반으로 한 합성 결함 이미지와 대응 mask를 공개하여, 연구자들이 모델 비교 실험 및 성능 평가에 활용할 수 있는 표준 데이터셋을 제공할 수 있다.

- 교육용 실습 자료 활용

대학, 연구기관 등에서 AI 기반 제조 기술 교육 시, 실제 결함 데이터를 다루기 어려운 문제를 해결하고, 가상의 데이터로 실습 가능한 교육 콘텐츠를 제공할 수 있다.